Explicação completa do código — Clínica (Grupo 3, Etapa 4)

Rafael de Oliveira Machado · POO · Etapa 4

Esta explicação cobre só o que foi feito na Etapa 4: `Agendavel`, `HorarioIndisponivelException`, `Atendimento`, `Consulta`, `Exame`, `CadastroPacientes`, `GerenciadorConsultas` e o `Main`. `Pessoa`, `Medico` e `Paciente` vêm das etapas anteriores e não foram alterados.

`Models/Agendavel.java` (interface)

Descreve a capacidade de ocupar a agenda de um médico.

- **Métodos abstratos**: `getData()`, `getMedico()` e `getDuracaoMinutos()`. Quem implementa é obrigado a fornecê-los.
- **`default boolean conflitaCom(Agendavel outro)`**: método com corpo dentro da interface, escrito uma vez e herdado por todos.
  1. Se os médicos forem diferentes (`!=`, comparação por referência), não há conflito.
  2. Calcula o início e o fim de cada um (`data.plusMinutes(duração)`).
  3. Dois intervalos se sobrepõem quando `inicioA < fimB` **e** `inicioB < fimA`.

Exemplo, consulta das 14:00 (14:00–14:30):

- Exame às 14:15 (14:15–14:35): `14:00 < 14:35` e `14:15 < 14:30`, então conflita.
- Exame às 14:30: `14:30 < 14:30` é falso, então não conflita. O fim é exclusivo, e encostar não é sobrepor.

`Models/HorarioIndisponivelException.java`

- **`extends Exception`**: é *checked*. O compilador obriga quem chama um método que a declara com `throws` a tratá-la com `try/catch` ou a repassar.
- **Construtor**: recebe a mensagem e a repassa com `super(mensagem)`. Depois `getMessage()` a devolve.
- **Regra**: o médico não pode ter dois atendimentos sobrepostos.

`Models/Atendimento.java` (classe abstrata)

Reúne o que é comum a Consulta e Exame. É `abstract` e `implements Agendavel`, então as subclasses também são `Agendavel`.

- **`enum Status`**: `AGENDADA`, `EM_ESPERA`, `REALIZADA`, `CANCELADA`. Evita strings digitadas errado.
- **Atributos**: `codigo`, `paciente`, `medico`, `data` e `status`.
- **Construtor `protected`**: valida código, paciente e médico (não aceita `null` nem vazio); `setData` valida a data; o status inicial é `AGENDADA`.
- **Métodos abstratos**: `getTipo()` devolve o nome do tipo ("Consulta", "Exame..."); `getDuracaoMinutos()` é redeclarado com `@Override` para deixar claro que também cumpre o contrato de `Agendavel`. Cada subclasse é obrigada a implementá-los.
- **Getters**: não há `setStatus` público. O estado só muda pelos métodos abaixo, que garantem que a transição faz sentido.
- **`entrarEmListaEspera()`**: vai para `EM_ESPERA`. Lança `IllegalStateException` se já estiver `REALIZADA` ou `CANCELADA`.
- **`confirmar()`**: vai para `AGENDADA` (quando sai da espera). Mesma proteção.
- **`realizar()`**: só funciona se estiver `AGENDADA`; passa para `REALIZADA`.
- **`cancelar()`**: não deixa cancelar o que já foi `REALIZADA`; senão vai para `CANCELADA`.
- **`toString()`**: usa `getTipo()` e `getDuracaoMinutos()`. É polimorfismo: cada objeto responde com o seu tipo.

`Models/Consulta.java`

- **`extends Atendimento`**: é um atendimento.
- **`DURACAO_MINUTOS = 30`**: constante (`static final`).
- **Construtor**: só repassa tudo para `super(...)`.
- **`getTipo()`** devolve "Consulta" e **`getDuracaoMinutos()`** devolve 30. São os abstratos implementados.
- **`realizar()` sobrescrito**: `super.realizar()` faz a validação e muda o status; `getPaciente().registrarConsultaRealizada()` soma 1 no contador. A regra mora aqui porque só consulta conta no contador. Exame não conta.

`Models/Exame.java`

- **Atributos próprios**: `nomeExame` e `exigeJejum`, que não existem em Consulta.
- **`DURACAO_MINUTOS = 20`**.
- **Construtor**: chama `super(...)`, valida que `nomeExame` não é vazio e guarda os campos.
- **`getTipo()`**: devolve "Exame (nome)", por exemplo "Exame (Eletrocardiograma)".
- **`getDuracaoMinutos()`**: devolve 20.

- Não sobrescreve `realizar()`, então usa o da superclasse.

`CadastroPacientes.java`

- **`Map<String, Paciente> pacientesPorCpf = new HashMap<>()`**: chave é o CPF (só dígitos), valor é o paciente. A busca é direta e não há chave repetida.
- **`cadastrar(Paciente)`**: se `containsKey(cpf)` for verdadeiro, lança `IllegalArgumentException` (CPF duplicado). Senão, `put`.
- **`buscarPorCpf(String)`**: normaliza o CPF recebido (aceita com ou sem pontos) e faz `get`. Devolve `null` se não achar.
- **`remover(String)`**: normaliza e faz `remove`. O `Map.remove` devolve o valor removido ou `null`, então `!= null` indica se removeu.
- **`total()`**: quantidade cadastrada.

`GerenciadorConsultas.java`

Três estruturas:

- **`agenda` (List)**: atendimentos confirmados, em ordem de entrada.
- **`listaEspera` (List)**: funciona como fila (FIFO): quem chegou primeiro é o primeiro a ser remanejado.
- **`codigosRegistrados` (Set)**: códigos únicos. O `Set.add()` devolve `false` se o elemento já existia.

`agendar(Atendimento)` — `throws HorarioIndisponivelException`

1. Se o código já está no Set, lança `IllegalArgumentException`.
2. `buscarConflito` procura na agenda algum conflitante.
3. Se achou, lança `HorarioIndisponivelException` com mensagem explicando quem conflita. O método termina e nada é adicionado.
4. Se não, registra o código, adiciona à agenda e imprime.

`entrarNaListaEspera(Atendimento)`

- `codigosRegistrados.add(...)` faz duas coisas: registra e verifica duplicidade (se der `false`, lança exceção).
- Muda o status para `EM_ESPERA` e adiciona na fila.

`buscarConflito(Atendimento novo)` (privado)

- Percorre a agenda. Um item conflita se `existente != novo` (não compara consigo mesmo), está `AGENDADA` e `novo.conflitaCom(existente)` é verdadeiro (a regra da interface).
- Devolve o primeiro conflitante, ou `null`.

`cancelar(Atendimento)`

1. `atendimento.cancelar()` vem primeiro. Se já foi realizado, lança `IllegalStateException` antes de mexer nas listas.
2. `agenda.remove(...)` devolve `boolean` dizendo se estava na agenda; também tira da espera, se estiver lá.
3. Se liberou vaga na agenda, chama `remanejarListaEspera()`.

`remanejarListaEspera()` (privado, com `Iterator`)

- Percorre a fila em ordem. Para cada item: se `buscarConflito(espera) == null`, ele agora cabe; `it.remove()` tira da espera, `confirmar()` volta para `AGENDADA`, e `agenda.add(...)` coloca na agenda.
- O conflito é recalculado a cada item, então não há sobreposição entre os promovidos.
- `Iterator` porque é a forma segura de remover durante o percurso. `list.remove()` num `for-each` lançaria `ConcurrentModificationException`.

`removerAtendimentosRealizados()`

- Outro `Iterator`, sobre a agenda: remove os `REALIZADA` e devolve quantos saíram.

`listarAgenda()`

- Imprime agenda e espera. `println(" " + a)` chama o `toString()` de cada um, e cada subclasse responde com o seu tipo (polimorfismo).

Getters

- Devolvem versões somente leitura (`Collections.unmodifiableList` / `Set`), então ninguém de fora altera as coleções internas.

`Main.java`

`tentarAgendar(gerenciador, atendimento)` (auxiliar)

- Chama `agendar` dentro de `try`.
- `catch (HorarioIndisponivelException e)`: mostra o erro e manda o atendimento para a lista de espera. É aqui que a exceção é tratada.
- `catch (IllegalArgumentException e)`: só mostra o erro, por exemplo código repetido, e não vai para a espera.

`main`, passo a passo:

1. **Criação**: `drCarlos`, `joao` e `maria`, que já existiam das etapas anteriores.
2. **Map**: cadastra os dois e mostra o total (2). A busca por "222.222.222-22" devolve Maria. Cenário inválido 1: "João Clone" com o mesmo CPF dá erro de CPF duplicado, lançado pelo `cadastrar`. Cenário inválido 2: CPF "123" dá erro no próprio construtor de `Paciente`.
3. **Agendamento** (dia 25/08/2026):
   - `C-001` João às 14:00: **agendada**.
   - `C-002` Maria às 14:00: mesmo médico e horário, conflita, **vai para a espera**.
   - `E-001` exame do João às 14:15: cai dentro da consulta das 14:00, conflita, **vai para a espera**.
   - `E-002` exame da Maria às 15:00: **agendado**.
4. **Set**: nova consulta com código `C-001` é rejeitada por código repetido (`IllegalArgumentException`) e não vai para a espera.
5. **Listagem**: agenda com C-001 e E-002; espera com C-002 e E-001; 4 códigos registrados.
6. **Cancelar C-001 (João)**: sai da agenda e o remanejamento roda. C-002 (Maria, 14:00) agora cabe e **entra na agenda**. E-001 (14:15) continua esperando, porque agora conflita com a consulta da Maria.
7. **Realizar**: Maria realiza C-002. Só consulta conta no contador do paciente, que passa a 1 (um exame não contaria).
8. **Cenário inválido**: cancelar C-002, já realizada, lança `IllegalStateException`, capturada e impressa.
9. **Iterator**: remove da agenda o que foi realizado (1 removido). Sobra o exame E-002 na agenda e E-001 na espera.

Limitação que aparece no passo 11: o `removerAtendimentosRealizados()` não chama o remanejamento. Por isso o E-001, que poderia caber agora, continua na espera. O remanejamento só roda em `cancelar`. Se o professor perguntar, é um ponto honesto de melhoria: chamar `remanejarListaEspera()` também ali.